TINDER SYSTEM DESIGN ARCHITECTURE

Scalable Location-Based Recommendation, Match Engine & Chat System

This document presents the high-level system design architecture for a large-scale dating application like Tinder, handling 10 Million Daily Active Users (DAU), real-time location matching, image storage, and low-latency 1-on-1 direct messaging.

Credits & Reference: Derived from System Design concepts by Rajneesh Verma, Avash Mitra, and Gaurav Sen (InterviewReady).

1. System Requirements & Scope

Prioritized Functional Requirements

- Profile Management: Store user metadata (name, age, gender, location, bio) and up to 5 profile images.
- Location-Based Recommendations: Recommend candidate profiles based on geographical proximity, age/gender preferences, and past swipe choices.
- Match Engine: Store mutual right-swipes as match records.
- Real-Time 1-on-1 Chat: Allow direct messaging between two users if and only if they have matched.

Requirements Not In Scope

- Profile moderation and automated content filtering.
- Payment subscriptions (Tinder Gold/Platinum).
- Enforcing daily swipe limits on free tier accounts.

2. Capacity & Scale Estimations

- *Active Users (DAU): 10,000,000 (10 Million users daily)*
- *Profile Photos: 5 photos per user @ 2 MB average file size*
- *Total Image Storage: 10M × 5 × 2 MB = 100 Terabytes (TB)*
- *Daily Matches: 0.1% of DAU = 1,000,000 matches/day (~1,000 matches/sec peak)*
- *Concurrent Chat Sockets: ~2,000,000 active persistent XMPP socket connections*

3. High-Level Architecture Components

Core Microservices & Subsystems

1. API Gateway Service: Handles load balancing, auth token validation (JWT), TLS termination, and rate limiting across internal services.
2. Distributed File System & CDN: Stores 5 profile images per user in Object Storage (S3/GCS) and serves them via edge CDN nodes for sub-100ms loading speeds.
3. Relational User DB (Geo-Sharded): Stores user metadata sharded horizontally by location with Master-Slave replication for failover.
4. Matcher Service & Database: Evaluates right-swipe events and records mutual matches in indexed tables.
5. Session Service & Redis Cache: Maps UserID to ConnectionID in Redis cache for instantaneous XMPP socket routing.
6. Recommendation Engine: Generates real-time candidate profile feeds based on location spatial indexes and age/gender filtering.

4. Architectural Trade-Off Analysis

Trade-Off 1: Image Storage (DFS + CDN vs Database BLOBs)

Storing images as BLOBs in relational databases offers mutability, transaction guarantees, and indexing. However, static image files do not need these features. Storing images in a Distributed File System (S3/GCS) backed by a CDN is significantly cheaper, faster, and highly scalable.

Trade-Off 2: Real-time Chat (XMPP Protocol vs HTTP Polling)

HTTP Polling requires clients to constantly send requests to check for new messages, draining battery and overwhelming servers. XMPP / WebSockets provide persistent peer-to-peer TCP channels, allowing server-pushed instantaneous messages with minimal overhead.

Trade-Off 3: Match Details (Server-Side DB vs Client-Side Storage)

Storing match records on the client saves server disk space, but match history is lost upon app reinstallation or cache clearance. Server-side relational database storage ensures an immutable, tamper-proof single source of truth.

5. Database Schema & Indexing

User Profiles Table (Location Sharded)

```
CREATE TABLE users (
    user_id VARCHAR(36) PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
    age INT NOT NULL,
    gender VARCHAR(20) NOT NULL,
    latitude DECIMAL(9,6) NOT NULL,
    longitude DECIMAL(9,6) NOT NULL,
    bio TEXT,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
CREATE INDEX idx_user_location ON users (latitude, longitude);
```

Match Engine Table

```
CREATE TABLE matches (
    match_id VARCHAR(36) PRIMARY KEY,
    user_id_1 VARCHAR(36) NOT NULL,
    user_id_2 VARCHAR(36) NOT NULL,
    is_matched BOOLEAN DEFAULT FALSE,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT unique_user_pair UNIQUE (user_id_1, user_id_2)
);
CREATE INDEX idx_user_matches ON matches (user_id_1, is_matched);
```

6. API Interface Specifications

Profile Service:
POST /user/signup -> Creates new user account & returns auth token
GET /user/login -> Authenticates user & issues session JWT
GET /user/:userID -> Gets target user profile details
PUT /user/:userID -> Updates location coordinates & bio

Matcher & Recommendation Service:
GET /recommendation -> Returns ranked candidates feed
POST /swipe -> Records user swipe action (LIKE / PASS)
GET /match -> Returns active mutual matches for logged-in user

Session & Chat Service:
POST /session/connection -> Establishes persistent socket mapping
GET /session/users/:id -> Resolves connected user pair metadata